LightPFN 1.5

Native categoricals, quantile regression, wide tables, and fast portable inference under 5 million parameters

Giorgio Ottoboni

Technical report, version 1.1, 11 November 2026

https://github.com/GioOtto/LightPFN

Abstract

We present LightPFN 1.5, an update of the LightPFN 1.0/1.1 tabular foundation model [1] that adds native categorical handling, a dedicated regression model, a synthetic prior mix with matched binary/multiclass task shares with long-context pretraining up to 60,000 rows, and a faster GPU inference path, while keeping each checkpoint below 4.8 million parameters and training exclusively on synthetic data. Rather than replacing the 1.1 B4 trunk with a heavier candidate architecture, which we show suffered from late retrieval-decoder temperature runaway and did not outperform B4 at equal training steps once stabilized, LightPFN 1.5 retains the B4 trunk and trains its 6,384-parameter categorical adapter end-to-end from initialization (4,609,472 parameters for classification). In pretraining, the adapter combines a dedicated categorical Fourier projection with CatBoost-style ordered target statistics and a 25% categorical mask-drop rate, paired with an automatic integer-category detector (`cat="auto"`) at inference. For regression, we audit the synthetic generator against real OpenML benchmarks, introduce the `r2` observation recipe (learnability filter, calibrated noise, heavier tails, and grid discretization), and fine-tune the pretrained long-context classification trunk with a 999-grid quantile head and fused moment retrieval (4,772,519 parameters). At the end of long-context pretraining (single estimator), the classifier preserves the 55-task OpenML-CC18 (D3) accuracy of 1.1.0 (mean AUC 0.9103 vs. 0.9109; +0.80 percentage points over default CatBoost, +1.96 over LightGBM, +1.99 over XGBoost on its 54 successful tasks) while lifting the 10,000-row high-cardinality categorical benchmark Dcat to 0.7075 (+0.43 points over default CatBoost, +0.27 over 1.1.0, and +1.79 over 1.0) and the 46-cell large-table suite D4 to 0.8200. A final 8,500-step fine-tuning on synthetic tables with 64 to 500 features, beyond the 100-feature limit of pretraining, improves the nine D3 datasets with more than 100 features (log loss -0.048 $[-0.108, -0.004]$) without measurable cost elsewhere (single-estimator D3 0.9102, Dcat 10k 0.7082), and raises the author-run TabArena-Lite classification Elo from 1416 to 1436 on the same RTX 5090 (1.1: 1423; the intervals overlap). In regression, two-member inference (y and $\log y$ or $-y$, up to 50,000 context rows) reaches 0.1748 normalized CRPS and 0.4042 normalized RMSE on the 23-task D3-reg benchmark (beating default CatBoost on 22 of 23 tasks) and reaches a local TabArena-Lite regression Elo of 1450 (vs. 1416 for default CatBoost, 1217 for LightGBM, and 1183 for XGBoost), though a substantial gap remains to larger foundation models. Compiler-free half-precision GPU inference with silent per-block FP32 fallback, shared weight views, and inference caches improves the geometric-mean classification training and prediction times by 1.89× and 2.04× relative to 1.1.0 on eight tasks under an 8-fold, 4-estimator protocol on the same RTX 5090. Prediction differences are small but nonzero. On the official TabArena leaderboard, where the maintainers ran `lightpfn` 1.5.1 on all 816 splits of the 51 datasets, LightPFN is 31st of 99 entries with Elo 1447 ($+53/-61$), above every tree-based entry including tuned and ensembled ones (intervals overlap), 28th of 103 in classification (1468) and 47th of 101 in regression (1431; default CatBoost 1410), and lies on the Pareto fronts of Elo against median training and prediction time. Code and weights are released under the Apache 2.0 license as the `lightpfn` 1.5 package (1.5.0, and 1.5.1 with inference fixes and faster GPU inference on the same weights).

1 Introduction

LightPFN 1.0 and 1.1.0 [1] showed that a 4.60-million-parameter in-context tabular classifier, pretrained from scratch on synthetic structural causal graphs and rule tasks, in the line of TabPFN [4] and TabICL [9], can

outperform default gradient-boosted decision trees (CatBoost [8], LightGBM [6], XGBoost [2]) on small-table classification benchmarks while running on a commodity CPU or consumer GPU. At the same time, the 1.1 study identified three concrete limitations:

1. **Categorical features.** LightPFN 1.1 treated categorical columns as ordinal numeric codes. On tables dominated by nominal or high-cardinality categoricals, it trailed default CatBoost, and post-hoc remedies on the frozen 1.1.0 weights (inference-time target encodings, frequency columns, or fine-tuning a categorical adapter with the trunk frozen) gave no gain on the 10,000- and 25,000-row categorical benchmark Dcat [1, Section 12].
2. **Regression.** LightPFN 1.1 supported classification only (2 to 10 classes). On the combined TabArena leaderboard, its 13 regression tasks had to be imputed.
3. **GPU inference latency.** While CPU inference used cache blocking and folded RMSNorm weights, GPU inference ran in 32-bit floating point without the new half-precision weight view and inference caches.

This report extends the original LightPFN report [1] and documents LightPFN 1.5, which addresses all three limitations under the same constraints as 1.1: each checkpoint remains around 4.8 million parameters (4.61M for classification, 4.77M for regression), pretraining uses synthetic data only with no distillation or external model outputs, and all design choices are made on development benchmarks disjoint from TabArena. We refer to the 1.1 report [1] for the base B4 trunk, the structural causal model (SCM) and rule generators, and the statistical evaluation harness.

Overview of changes. LightPFN 1.5 introduces five main changes: (1) a 6,384-parameter categorical adapter trained from random initialization with 25% categorical mask-drop and paired with an automatic integer-category detector `cat="auto"` (Section 2); (2) a classification prior with matched binary/multiclass task shares (90% graph v4 + 10% rule v1 at binary task probability $p_{\text{bin}} = 0.55$) after controlled experiments showed that earlier graph v4 multiclass deficits were a class-share confound ($p_{\text{bin}} = 0.78$ vs. 0.55), alongside an audited regression prior recipe `r2` with a structural learnability filter (Section 3); (3) retention of the B4 trunk after diagnosing and fixing a retrieval-temperature runaway in a candidate 8-block architecture (A3) that still failed to beat B4 in a paired 30,000-step gate (Section 4); (4) a five-stage training schedule (120,000 main + 15,000 long-context classification steps up to 60,000 rows, an 8,500-step wide-table classification fine-tuning up to 500 features, and 39,063 main + 5,000 long-context regression fine-tuning steps for a separate 999-quantile head; Figure 2, Section 5); and (5) compiler-free FP16 GPU inference (`precision="auto"`) with silent per-block FP32 fallback, shared half-precision weight views, and hand-fused PyTorch kernels (Section 9).

2 Model Changes vs. 1.1

LightPFN 1.5 retains the five-stage B4 trunk of [1, Section 2] (4,603,088 parameters): (1) a 64-dimensional cell embedder (`CellEmbedder`); (2) a 2-layer inducing-point set attention (ISAB, 64 inducing tokens) column stage (`ColumnStage`); (3) three rounds of summary-mediated row refinement ($R = 3$, $K = 4$ summary tokens) followed by a second independent ISAB column stage; (4) a 3-layer row compression stage with 4 learned readout tokens concatenated into a 256-dimensional row vector; and (5) a 7-block in-context learning (ICL) transformer ($d = 256$, 8 query heads, 1 key/value head for query rows, 16 thinking tokens, and $\log n$ softmax scaling) followed by a 4-head retrieval attention decoder over 16 label slots. Figure 1 shows the trunk and the two architectural additions of 1.5, described below: the categorical adapter (panels A and B), used by both checkpoints, and the regression modules (panels C and D).

2.1 Categorical Adapter

When `Config.cat_adapter=True` is enabled, the model adds 6,384 parameters (4,609,472 total for classification) split between the cell embedder and the first column stage (`lightpfn/model/lightpfn.py`, `lightpfn/model/catstats.py`):

1. **Dedicated categorical Fourier embedding.** In `CellEmbedder`, each cell groups its own column and two circular neighbors at offsets $o \in \{0, 1, 3\}$. Each member contributes its soft-clipped standardized value

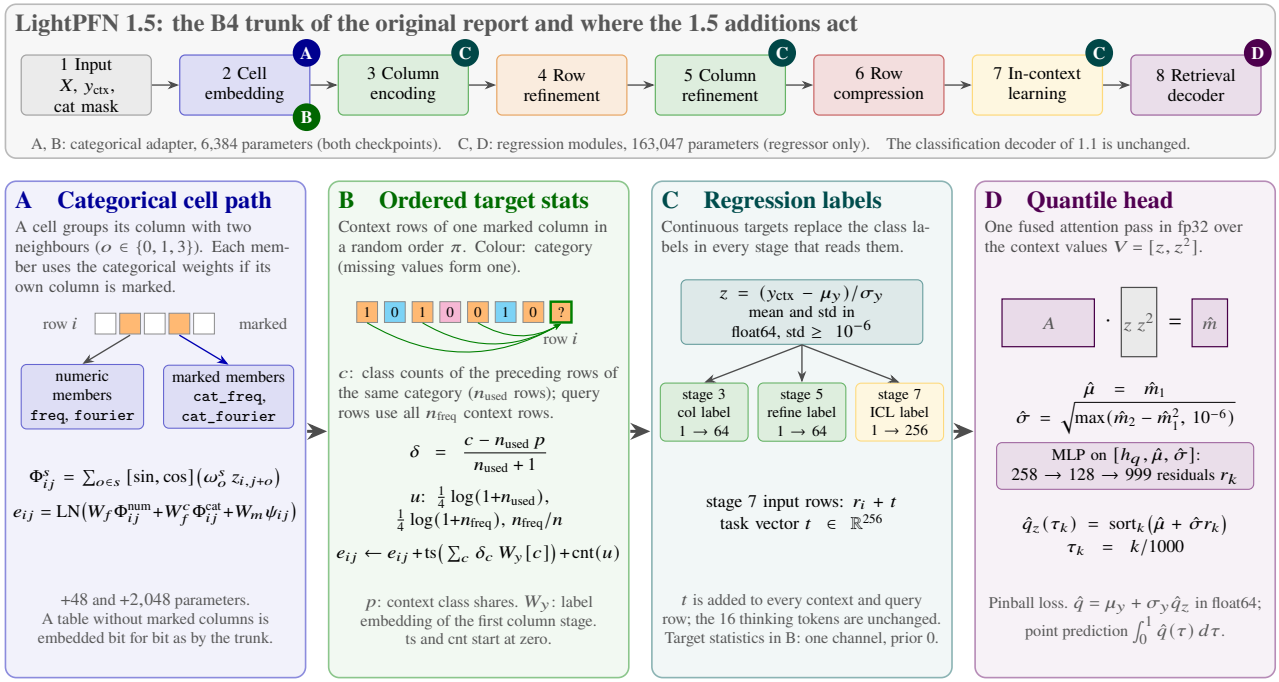


Figure 1. LightPFN 1.5 architecture. Top: the eight operators of the B4 trunk of the original report, unchanged in 1.5, with the points where the additions act. **A**: categorical members of a cell group use their own Fourier frequencies and projection. **B**: ordered target statistics of each categorical cell, embedded through the label embedding of the first column stage and added to the cell embeddings. **C**: in the regressor, standardized targets replace the class-label embeddings in the column, column-refinement and ICL stages, and a task vector is added to every row entering the ICL stage. **D**: the retrieval decoder returns the first two moments of the context targets, from which a small MLP predicts 999 sorted quantiles. A table without categorical columns is processed exactly as by the trunk alone.

$z = 5\tanh(((x-\mu)/(\sigma+10^{-6}))/5)$, expanded through 16 learned frequencies for that offset. The three sine/cosine vectors are summed before a shared bias-free projection $\mathbb{R}^{32} \to \mathbb{R}^{64}$. A separate projection embeds grouped standardized values, missingness flags, and rank Fourier features. With `cat_adapter=True`, a parallel set of learned frequencies `cat_freq` ($3 \times 16 = 48$ parameters) and a bias-free projection `cat_fourier` ($\mathbb{R}^{32} \to \mathbb{R}^{64}$, 2,048 parameters) are added. Group members with boolean categorical mask $\text{cat}_j = 1$ route through `cat_freq` and `cat_fourier`, while columns with $\text{cat}_j = 0$ use the numerical path.

2. **Ordered target statistics and count features.** In `ColumnStage`, each categorical cell (i, j) receives a vector of category statistics computed from the context rows $(X_{\text{ctx}}, y_{\text{ctx}})$. Missing values (`NaN`) form a distinct category key of their own, and categories unseen in the context receive zero features. For its category-specific counts, a context row i excludes its own label: given a random permutation π of the n context rows, context row i uses only the $n_{\text{used}}(i, j)$ context rows of the same category that precede it in π (CatBoost's ordered target statistics [8]). The global classification prior p below still uses the full context class proportions. Leave-one-out target statistics were tested during 1.1 development and caused a 12-point AUC collapse because leave-one-out statistics can encode a row's excluded label through category totals [1, Section 12]. Query rows q use all $n_{\text{used}}(q, j) = n_{\text{freq}}(q, j)$ context rows belonging to their category, looked up via binary search against a per-category summary table cached during `encode`.
   For a C-class classification task with empirical context class prior $p \in \mathbb{R}^C$ and smoothing parameter $\alpha = 1.0$ (`Config.cat_smooth`), let $c(i, j) \in \mathbb{R}^C$ be the class counts among the $n_{\text{used}}(i, j)$ preceding context rows of cell (i, j)'s category, and let $n_{\text{freq}}(i, j)$ be the total number of other context rows in that category (n_{used} for query rows). The smoothed class-distribution deviation and the three count features are

$$\delta(i,j) = \frac{c(i,j) - n_{\text{used}}(i,j)\,p}{n_{\text{used}}(i,j) + \alpha}, \qquad u(i,j) = \left[\tfrac{1}{4}\log(1 + n_{\text{used}}(i,j)),\ \tfrac{1}{4}\log(1 + n_{\text{freq}}(i,j)),\ \frac{n_{\text{freq}}(i,j)}{n}\right]. \quad (1)$$

   The module `CatStatsEmbed` maps $\delta(i, j)$ through the column stage label embeddings $W_y \in \mathbb{R}^{S\times 64}$ and

a learned bias-free matrix ts : $\mathbb{R}^{64} \to \mathbb{R}^{64}$ (4,096 parameters), and maps $u(i, j) \in \mathbb{R}^3$ through a bias-free projection cnt : $\mathbb{R}^3 \to \mathbb{R}^{64}$ (192 parameters). Both projections start at zero. At initialization (`cat_freq` = `freq`, `cat_fourier` = `fourier`, ts = 0, cnt = 0), the adapter initially reproduces the plain model up to floating-point rounding; when pretrained from scratch, all 6,384 adapter parameters are learned jointly with the trunk.

2.2 Automatic Categorical Detection and Pretraining Mask-Drop

In scikit-learn workflows, users often pass NumPy arrays where categorical columns have already been ordinal-encoded as integers, without supplying a pandas `category` dtype or an explicit `cat` mask. If a model is pretrained only with ground-truth masks, passing an unmasked integer-coded categorical column at test time routes it through the numerical cell path, which has never seen discrete category codes without adapter statistics. We combine two mechanisms to handle this:

1. **Inference-time detection (`cat="auto"`).** The helper `auto_categorical(X)` (`lightpfn/sklearn.py`) marks any pandas `category`, string, object, or boolean column as categorical, and flags any numeric column whose non-missing training values are all exact integers with between 2 and 20 distinct values. On an 8,000-step V15 checkpoint, omitting the categorical mask (`cat=None` vs. ground-truth OpenML masks) costs −0.21 AUC points [−0.43, −0.04] across D3 (−0.63 points on its 18 categorical tasks) and −3.04 points [−5.83, −0.99] on Dcat 10k. Enabling `cat="auto"` reduces the gap to ground-truth masks to −0.07 points [−0.21, +0.04] on D3 and −0.39 points [−0.93, +0.08] on Dcat 10k; false positives on the 21 numerical D3 tasks with integer columns have a mean effect of −0.008 points. These are exploratory single-checkpoint measurements.
2. **Pretraining categorical mask-drop (`--cat-mask-drop 0.25`).** During classification pretraining, for each task independently, we clear its entire categorical mask with probability $p_{\text{drop}} = 0.25$ before the forward pass. This forces the numerical cell path and trunk to process categorical columns in 25% of tasks (including high-cardinality columns with > 20 levels that `cat="auto"` cannot distinguish from continuous integers) without adapter statistics. In a 2,000-step A/B test, $p_{\text{drop}} = 0.25$ shrinks the no-mask penalty (`none` minus `true`) from −0.41 to −0.04 points on D3 and from −3.94 to −0.33 points on Dcat 10k. In a paired 8,000-step confirmation, $p_{\text{drop}} = 0.25$ vs. $p_{\text{drop}} = 0$ shows no clear cost when the true mask is provided (+0.03 points on D1, +0.05 points [−0.19, +0.27] on D3, and −0.03 points [−0.46, +0.36] on Dcat 10k) while improving unmasked D3 AUC by +0.34 points [−0.03, +0.77].

2.3 Quantile Regression Head and Two-Member Wrapper

To support regression without altering the classification trunk, `Config(head="quantiles")` adds a `RegressionModules` bundle of 163,047 parameters (`lightpfn/model/regression.py`), bringing the regression checkpoint to 4,772,519 parameters:

1. **Target normalization and label embeddings.** Given context targets $y_{\text{ctx}} \in \mathbb{R}^{B\times n}$, the model computes their mean μ_y and population standard deviation in `float64`, floors the standard deviation at 10^{-6}, and returns standardized targets $z = (y_{\text{ctx}} - \mu_y)/\sigma_y$ in `float32`. In the categorical adapter, ordered target statistics use a fixed zero prior ($\mu_z = 0$) and $C = 1$ channel so a context row's own target never leaks through an empirical prior. Continuous context targets are injected into the first column stage via `col_label` (Linear(1, 64), 128 parameters), into the second column stage via `refine_label` (Linear(1, 64), 128 parameters), and into the ICL stage via `icl_label` (Linear(1, 256), 512 parameters), and a learned regression task token `task` $\in \mathbb{R}^{256}$ (256 parameters) is added to every context and query row at the input of the ICL stage (the 16 thinking tokens are unchanged).
2. **Fused retrieval moments and 999-quantile residual head.** At the end of the ICL stage, the retrieval decoder projects the normalized context representations to keys $K \in \mathbb{R}^{B\times H\times n\times d_h}$ ($H = 4, d_h = 16$) and the scaled query representations to queries $Q \in \mathbb{R}^{B\times H\times M\times d_h}$. Rather than running two attention passes, `RetrievalDecoder.moments` concatenates $V = [z, z^2] \in \mathbb{R}^{B\times n\times 2}$ and executes a single fused scaled dot-product attention pass in `float32` to retrieve the local first and second moments $\hat{m}_1, \hat{m}_2 \in \mathbb{R}^{B\times M\times 1}$ averaged

across the 4 heads:

$$\hat{\mu} = \hat{m}_1, \qquad \hat{\sigma} = \sqrt{\max\left(\hat{m}_2 - \hat{m}_1^2,\ 10^{-6}\right)}. \tag{2}$$

The `QuantileHead` (162,023 parameters: Linear$(258, 128) \to$ GELU $\to$ Linear$(128, 999)$) takes $[h_{\text{query}}, \hat{\mu}, \hat{\sigma}] \in \mathbb{R}^{B\times M\times 258}$ and predicts $G = 999$ raw scores $r \in \mathbb{R}^{B\times M\times 999}$ for the fixed quantile grid $\tau_k = k/1000$ ($k = 1, \ldots, 999$). The MLP outputs unconstrained residuals r_k. The model shifts and scales them by the retrieved moments, then sorts along the quantile axis:

$$\hat{q}_z(\tau_k) = \text{sort}_k(\hat{\mu} + \hat{\sigma} r_k), \qquad k = 1, \ldots, 999. \tag{3}$$

The output layer uses the standard linear-layer initialization; there is no softplus-increment parameterization or normal-quantile initialization. Predictions are unstandardized in `float64` via $\hat{q}(\tau_k) = \mu_y + \sigma_y \hat{q}_z(\tau_k)$. Training minimizes mean pinball loss over the 999 levels in standardized units, approximating CRPS$/(2\sigma_y)$.

3. **Two-member inference wrapper.** At inference (`V15Regressor` in `lightpfn/regressor.py`), the wrapper defaults to $n_{\text{estimators}} = 2$ and max_context = 50,000. The two members are encoded and queried in a single batched forward pass ($B = 2$): member 1 fits the raw target y, while member 2 fits $\log(y)$ when $\min(y) > 0$ and the sample skewness of the context targets is at least 2 (the `log_skew=2.0` default, Section 7.3; mapping predicted quantiles back via $\exp(\hat{q})$, clipped to the interval $[\max(2y_{\min} - y_{\max}, 0), 2y_{\max} - y_{\min}]$) or $-y$ otherwise (inverting and reversing the quantile grid via $-\hat{q}_{1-\tau}$). The two monotone quantile curves are averaged (L_2 Wasserstein barycenter), and the point prediction for squared-error metrics (RMSE, R^2) is the trapezoidal integral $\int_0^1 \hat{q}(\tau)\, d\tau = \mathbb{E}[Y \mid X]$ over $\tau_1, \ldots, \tau_{999}$ with constant tail extrapolation on $[0, 0.001]$ and $[0.999, 1]$.

3 Prior and Data

3.1 Graph v4 vs. Graph v3 and the Class-Share Confound

In the 1.1 report, replacing the graph v3 prior with graph v4 (which reduces input outlier clipping and adds Zipf-quantized categorical columns up to 256 levels) appeared to hurt multiclass accuracy in short 500-step and 8,000-step tests [1, Section 4]. An audit of the generated task pools revealed a class-share confound: the graph v3 generator used a binary-task probability of $p_{\text{bin}} = 0.55$ (leaving 45% multiclass tasks with 3 to 10 classes), whereas the graph v4 generator defaulted to $p_{\text{bin}} = 0.78$ (only 22% multiclass tasks, or 27% after mixing with rule tasks), while 28 of the 55 D3 tasks (51%) are multiclass.

To isolate the graph generator from the class distribution, we ran a controlled six-arm, 2,000-step experiment from random initialization (64 tasks/step, 90% graph + 10% rule) comparing $p_{\text{bin}} \in \{0.55, 0.78\}$ and graph v3 vs. graph v4 on seed 0, with the $p_{\text{bin}} = 0.55$ graph comparison repeated on seed 1:

- **Effect of p_{bin} within the same graph family.** Raising p_{bin} from 0.55 to 0.78 degrades multiclass AUC in both priors: on D3 multiclass tasks, $p_{\text{bin}} = 0.78$ minus 0.55 is -0.61 points raw (-1.06 EMA) for graph v3 and -1.26 points raw (-1.56 EMA) for graph v4, while binary D3 tasks are unchanged (+0.14 and +0.11 points raw, within noise). On D1 multiclass tasks, $p_{\text{bin}} = 0.78$ similarly loses -0.49 points raw (-0.80 EMA) for v3 and -1.05 points raw (-1.28 EMA) for v4.
- **Graph v4 vs. graph v3 at equal class share ($p_{\text{bin}} = 0.55$).** When both generators use $p_{\text{bin}} = 0.55$, the multiclass gap vanishes: across seeds 0 and 1, graph v4 minus graph v3 is +0.005 and +0.043 points raw on D1, and -0.041 points $[-0.25, +0.19]$ and -0.012 points $[-0.24, +0.29]$ on D3. At the same time, on the 12 categorical cells of Dcat (6 datasets at 10,000 and 25,000 rows), graph v4 at $p_{\text{bin}} = 0.55$ beats graph v3 on both seeds: +0.22 points $[+0.10, +0.34]$ raw (10/12 cells; +0.18 EMA) on seed 0 and +0.12 points $[+0.004, +0.23]$ raw (9/12 cells; +0.06 EMA) on seed 1.

LightPFN 1.5 classification pretraining therefore uses 90% graph v4 and 10% rule v1 with $p_{\text{bin}} = 0.55$ in both families. These are matched binary/multiclass task shares, not equal frequencies for all class counts.

3.2 High-Cardinality Lookup Prior: Tried and Dropped

Because graph v4 categorical columns are formed by Zipf-quantizing continuous latent variables (up to 256 levels), we tested whether adding a dedicated synthetic family of high-cardinality lookup tables (up

to 1,000 levels with random per-level target effects, constrained to at least eight rows per level on average, `lightpfn/prior/lookup.py`) would further improve performance on high-cardinality categorical datasets. In an 8,000-step 2×2 factorial experiment comparing the plain B4 trunk against V15 (B4+categorical adapter), each trained on the same group-weighted mix (67.5% short graph v4, 22.5% longer graph v4, 10% rule at $p_{\text{bin}} = 0.55$), with the lookup arms augmenting graph tasks with a lookup column at probability 20%:

- **Architectural effect (V15 minus B4 on the base prior, EMA 8k).** Adding the categorical adapter improves D1, D3, Dcat, and the marked lookup probe; D2 falls from 0.9198 to 0.9173: D1 by +0.31 points $[+0.10, +0.53]$ (0.8569 vs. 0.8538), D3 by +0.24 points $[+0.01, +0.48]$ (0.8989 vs. 0.8965), Dcat 10k by +2.30 points (0.6939 vs. 0.6708, winning 5 of 6 datasets), Dcat 25k by +2.12 points (0.6982 vs. 0.6770, 5 of 6), and the synthetic high-cardinality lookup probe by +9.28 points averaged over cardinalities ($K = 1024$: 0.7115 vs. 0.6588, a +5.27-point gain).
- **Prior effect (V15 with lookup prior minus V15 on base prior, EMA 8k).** Adding the lookup prior degrades D1 by −0.48 points $[-0.68, -0.29]$ (0.8521 vs. 0.8569) and D3 by −0.17 points $[-0.37, +0.06]$ (0.8972 vs. 0.8989), fails to improve Dcat (−0.27 points at 10k, 0.6912 vs. 0.6939; −0.10 points at 25k, 0.6972 vs. 0.6982), and adds only +0.92 points on the lookup probe itself.

The categorical adapter already learns to exploit per-level target statistics from the Zipf-quantized categorical columns of graph v4; adding these lookup columns reduced D1 accuracy without improving Dcat in this one-seed test, so we dropped the lookup prior.

3.3 Regression Prior `r2` and Learnability Audit

Our initial synthetic regression generator (P0, `lightpfn/prior/regression.py` and `observation.py`) extracted a continuous target node from the SCM, standardized its location on 2,048 disjoint calibration rows, and passed it through one of nine observation channels (Gaussian, heteroscedastic Gaussian, Student-t, sinh-arcsinh, positive log-location, bimodal mixture, zero-inflated hurdle, Poisson-gamma counts, or top-coded censoring). Auditing 1,024 P0 tasks against the 23 real OpenML regression datasets of D3-reg exposed three mismatches: (1) *unlearnable tasks:* on 36.3% of P0 tasks, a 200-tree ExtraTrees regressor with `min_samples_leaf=2` achieved held-out $R^2 \le 0$ on a 70/30 split (median $R^2 = 0.145$), vs. only 4.3% (1 of 23 tasks) on D3-reg (median $R^2 = 0.776$); (2) *excessive channel noise and weak nonlinear signal:* only 15.5% of P0 tasks had $R^2_{\text{ET}} - R^2_{\text{Ridge}} > 0.10$, vs. 34.8% on D3-reg; and (3) *lack of discrete/rounded targets and heavy tails:* 43.5% of D3-reg datasets have unique-value ratio < 0.5 vs. 6.6% in P0, and 90th-percentile target skewness (4.51 vs. 2.80) and excess kurtosis (43.49 vs. 18.13) were higher in real data.

We addressed these in the `r2` regression prior recipe (`--reg-factors` set to `filter`, `noise`, `tails`, `discrete`):

- `filter`: fits a 32-tree ExtraTrees regressor on the 2,048 independent calibration rows to predict the latent SCM location from the features, and rejects structural drafts with OOB $R^2 < 0.10$ while retaining 5% of failing drafts. Observed noisy targets are not used in this filter.
- `noise` and `tails`: tightens additive noise to $[0.02, 1.0]$ and log-scale noise to $[0.03, 0.70]$, links hurdle zero probabilities to the latent location via a logistic link, reduces the bimodal gap parameter to $[0.3, 1.5]$, widens positive log-spread to $[0.3, 1.5]$, and upweights skewed/count/censored channels.
- `discrete`: with probability 0.35 on eligible continuous channels, rounds targets to a grid whose step is sampled log-uniformly from $[0.05, 1.0]$ times a declared channel-dependent unit.

In 2,000-step A/B tests across two seeds, `r2` on graph v3 lowered D3-reg raw normalized CRPS by −4.7% $[-7.5\%, -2.5\%]$ on seed 0 (19/23 tasks) and −3.4% $[-6.9\%, -1.1\%]$ on seed 1 (19/23 tasks), for a two-seed mean reduction of −4.1% $[-7.0\%, -1.9\%]$ (21/23 tasks), outperforming `r2` on graph v4 (−3.3% mean across seeds). We therefore selected `r2` on graph v3 for regression pretraining (noting in Section 7.3 a subtle interaction with categorical cardinality in graph v3 uncovered after training).

4 Negative Result: The Candidate "v2" Architecture (A3)

Before adopting the B4-based LightPFN 1.5 design, we investigated a redesigned 4.8-million-parameter candidate trunk, denoted A3 (or C2v2), which used 8 distinct (unshared) ICL blocks with a reduced MLP ratio

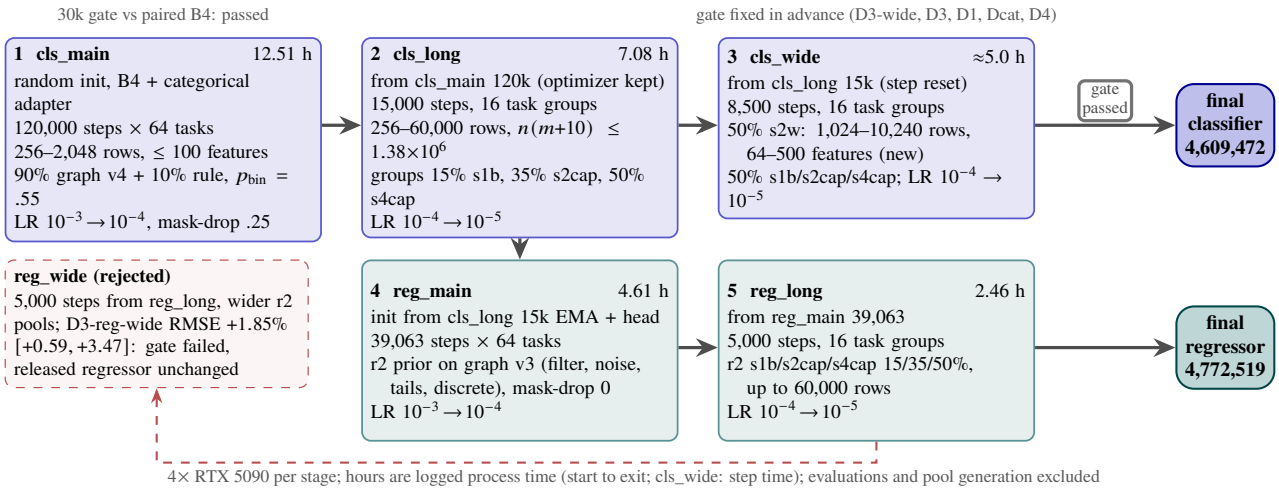


Figure 2. Training pipeline of LightPFN 1.5. Each box gives the initialization, steps, data and learning-rate schedule of one stage, with the logged hours on 4× RTX 5090. The final classifier comes from the wide-table fine-tuning, which passed a gate declared before its evaluation; the final regressor is the end of the long-context regression stage. A wide-table regression fine-tuning (dashed) failed its gate and was not released (Section 7.2).

to stay under 5 million parameters, a heavier cell frontend with an explicit level-ID embedding view, and an unshared retrieval decoder.

Late training instability and decoder diagnostics. While A3 performed well in 8,000-step screens, it became unstable in 30,000-step pretraining runs: at peak learning rate $5 \cdot 10^{-4}$ it diverged between 9,500 and 15,000 steps, and at 10^{-3} it completed 30,000 steps but fell far behind the v1 30,000-step checkpoint (D1 −2.83 points, D2 −5.20 points, D3 −0.95 points). A replay of nine checkpoints across 1,728 forward/backward replays documented two growing decoder scale components in the retrieval decoder: (1) in `SoftmaxScaling`, the effective scaling base at context length $n = 1024$ grew from 10.9 at 7,500 steps to 1,246 at 30,000 steps (vs. 1.6 in v1), while the participation ratio of the 64-dimensional diagonal query/key metric collapsed from 42 dimensions to 5; and (2) in the original unbounded run, the product of the learned RMSNorm affine gains on the retrieval queries and keys grew from 1.05 to 7.93. Post-hoc interventions on the damaged 30,000-step checkpoint (temperature clipping, FP64 decoding, or removing affine gains at inference) could not recover v1 accuracy, consistent with coadaptation of the decoder and upstream representations; the replay does not establish the original cause.

Stabilized 30,000-step paired gate. Setting `decoder_scale_mode="atan"` and removing the retrieval query/key affine weights (`Config.retrieval_qk_affine=False`, bounding retrieval logits by $|\text{logit}| \le 64$ by construction) produced a stable 30,000-step run; it does not establish stability beyond that budget. In a pre-declared, paired 30,000-step gate comparing B4 against stabilized A3 on the exact same data stream (64 tasks/step, graph v4 + rule at $p_{bin} = 0.55$), A3 trained with zero non-finite gradients for all 30,000 steps, but failed the quality gate against B4 on EMA weights: D3 AUC was −0.14 points $[-0.26, -0.03]$ (0.9043 vs. 0.9057), D1 was −0.54 points $[-0.77, -0.32]$ (0.8664 vs. 0.8718, losing −1.94 points on rule tasks), D2 was −1.21 points (0.9235 vs. 0.9356), and CPU inference at 100 columns was 1.6–1.7× slower than B4. Meanwhile, B4 with the 6,384-parameter categorical adapter at 8,000 steps already exceeded A3's 30,000-step Dcat 10k score (0.6939 vs. 0.6864). We therefore abandoned A3 and built LightPFN 1.5 on the B4 trunk.

5 Training Recipe and Selection Protocol

5.1 Training Stages

LightPFN 1.5 is trained in five stages (Figure 2): three for the classifier (main pretraining, long-context training and a wide-table fine-tuning) and two regression fine-tuning stages starting from the long-context classifier, all using data-parallel training across four NVIDIA RTX 5090 GPUs per stage with `bfloat16` mixed precision, Muon [5] for internal block matrices, AdamW ($\beta_1 = 0.9, \beta_2 = 0.98$, weight decay 0.01) for the remaining

parameters, gradient clipping at 1.0, and EMA decay 0.999:

1. **Main classification stage (`cls_main`, 120,000 steps).** Initialized from random weights with B4 + `cat_adapter=True` (4,609,472 parameters). Trains for 120,000 steps at 64 tasks per step (7,680,000 task draws across 16 streamed pool pairs of 252,160 generated tasks each, 4.03×10^6 unique tasks, a two-pass sizing budget with a 5% generation margin) on 90% graph v4 + 10% rule v1 ($p_{\text{bin}} = 0.55$, 256–2,048 rows, ≤ 100 features, `--cat-mask-drop 0.25`, warmup 2,000, cosine LR $10^{-3} \rightarrow 10^{-4}$, 800,000 cells/GPU).
2. **Long-context classification stage (`cls_long`, 15,000 steps).** Resumes from `cls_main` step 120,000 (warmup 200, cosine LR $10^{-4} \rightarrow 10^{-5}$, `--cat-mask-drop 0.25`). Each step draws 16 task groups across the 4 GPUs (an expected 57.6 tasks/step and 864,000 task draws) from three row-count regimes (90% graph v4 + 10% rule at $p_{\text{bin}} = 0.55$): 15% from `s1b` (256–2,048 rows, 8 tasks/group), 35% from `s2cap` (1,024–10,240 rows, 4 tasks/group, with the preset feature cap), and 50% from `s4cap` (4,096–60,000 rows, 2 tasks/group, capped at 800,000 cells). Micro-batches enforce $n(m + 10) \leq 1.38 \times 10^6$ with `--fit-rows`. These row-count ranges refer to the whole synthetic table, including context and query rows. The 15,000-step EMA checkpoint (`cls_long/ema_step015000.pt`) initializes the regression stages and the wide-table fine-tuning below.
3. **Main regression fine-tuning stage (`reg_main`, 39,063 steps).** Two pilot tests guided this stage: first, in a 2,000-step, two-seed pilot comparing the 999-grid `quantiles` head against a distributional `transport` head, `transport` had +18.0% [+7.5%, +31.8%] worse normalized CRPS on D3-reg, so `quantiles` was chosen; second, warm-starting V15 regression from a pretrained classification checkpoint reduced 2,000-step D3-reg CRPS by −10.1% raw (−15.5% EMA: 0.2122 vs. 0.2490) relative to training from scratch, matching 18,000 steps of scratch training in 2,000 steps and reaching 0.1975 EMA CRPS at 10,000 steps. We therefore initialize `reg_main` from the final `cls_long` EMA checkpoint (`--init`) and fine-tune the network with 4,772,519 stored parameters for 39,063 steps ×64 tasks/step = 2,500,032 task draws (13 streamed pools of 105,216 `r2` graph v3 tasks each, 1.37×10^6 unique tasks, warmup 200, cosine LR $10^{-3} \rightarrow 10^{-4}$, 400,000 cells/GPU, `--cat-mask-drop 0`).
4. **Long-context regression stage (`reg_long`, 5,000 steps).** Resumes from `reg_main` step 39,063 for 5,000 steps (warmup 200, cosine LR $10^{-4} \rightarrow 10^{-5}$, 16 groups/step, an expected 288,000 task draws across 15% `r2_s1b`, 35% `r2_s2cap`, and 50% `r2_s4cap`, $n(m+10) \leq 1.38 \times 10^6$ with `--fit-rows`). Both regression stages enable `LIGHTPFN_NAN_GUARD`, which captures diagnostics and aborts before an update on non-finite gradients or a non-finite gradient norm; it does not zero gradients or skip bad updates silently. All 44,063 production regression steps completed without a recorded guard failure. An earlier short pilot had a non-finite-gradient failure that was not reproduced in the subsequent diagnostic replays. The 5,000-step EMA checkpoint (`reg_long/ema_step005000.pt`) is the final LightPFN 1.5 regressor.

Wide-table classification fine-tuning (`cls_wide`, 8,500 steps). Pretraining never generated more than 100 features, and a comparison of 1.5 with 1.1.0 found one repeated weakness: tables with more than 60 to 100 features. We therefore continue the `cls_long` checkpoint (model, EMA and optimizer state, step counter reset) for 8,500 steps with the same optimizer settings (warmup 200, cosine LR $10^{-4} \rightarrow 10^{-5}$, `--cat-mask-drop 0.25`, $n(m + 10) \leq 1.38 \times 10^6$) and fresh pools of the same generators: 8% `s1b`, 17% `s2cap`, 25% `s4cap`, and 50% `s2w`, a new pool with 1,024–10,240 rows and 64–500 features (at most 10^6 cells). The step count was fixed from the measured step time before any quality result, and the gate was declared in advance: on the final EMA only, D3-wide (the nine D3 datasets with more than 100 features, all columns) must improve in AUC or log loss with the 95% interval excluding zero, D3 must not lose more than 0.10 points, and D1, Dcat and D4 must not worsen beyond noise; only then is TabArena-Lite run. The final EMA (`cls_wide/ema_step008500.pt`, about 5.0 h of logged step time on 4× RTX 5090) passes the gate and is the final LightPFN 1.5 classifier (Section 6.2).

Compute budget. Each stage used a node with 4× NVIDIA RTX 5090 GPUs (32 GiB VRAM each) and an AMD EPYC 9654 CPU. Main classification and the remaining stages ran on two successive rented nodes; CPU pool generation overlapped training. Summing the logged start-to-exit intervals of the training processes gives 12.51 h for `cls_main`, 7.08 h for `cls_long`, 4.61 h for `reg_main`, and 2.46 h for `reg_long`: 26.7 h. The wide-table fine-tuning was paused twice for milestone evaluations; the sum of its logged step times is 5.01 h.

Model / stage	steps	D1	D2	D3 AUC	D3 log loss	Dcat 10k	Dcat 25k	Scale	D4
CatBoost (default)	–	–	–	.9023	.4012	.7031	.7118	.9071	.8189
LightGBM (default)	–	–	–	.8907	.5174	.6809	.6829	–	.7848
XGBoost (D3: 54 tasks)	–	–	–	.8889	.5213	.6421	.6597	–	.7903
LightPFN 1.0 (v1 30k)	30k	.8703	.9373	.9054	.3807	.6706	.6733	–	–
LightPFN 1.0 (`final`)	120k	.8781	.9331	.9104	.3586	.6895	.6935	.9025	.8063
LightPFN 1.1.0 (`final_long`)	159.25k	.8783	.9391	.9109	.3550	.7047	.7098	.9097	.8188
1.5 `cls_main`	30k	.8713	.9359	.9054	.3763	.6703	–	–	–
1.5 `cls_main`	60k	.8744	.9444	.9074	.3677	.6766	–	–	–
1.5 `cls_main`	90k	.8775	.9392	.9098	.3610	.7025	–	–	–
1.5 `cls_main`	120k	.8779	.9485	.9094	.3594	.7016	–	–	–
1.5 `cls_long` (+5k)	125k	.8772	.9481	.9103	.3586	.7078	–	–	–
1.5 `cls_long` (+10k)	130k	.8774	.9474	.9104	.3583	.7056	–	–	–
1.5 `cls_long` (+15k, final)	135k	.8777	.9531	.9103	.3582	.7075	.7134	.9102	.8200

Table 1. Classification benchmark progression across training milestones (mean ROC AUC, and mean log loss on D3; single estimator, full context). D1: 1,536 held-out synthetic tasks; D2: 125 mechanism probes; D3: 55 OpenML-CC18 tasks outside TabArena; Dcat 10k / 25k: 6 high-cardinality categorical datasets at 10,000 and 25,000 training rows; Scale: 162 splits across 15 datasets (1k–94,320 training rows); D4: 46 splits across 13 large datasets (10k–100k rows). The released LightPFN 1.0 row uses the 120k-step checkpoint on all suites. XGBoost D3 excludes its failed task (54 tasks); the other D3 entries use 55.

In total, approximately **31.7 hours** or **127 RTX 5090 GPU-hours**. These intervals include process startup and saving (except for `cls_wide`), but exclude separate evaluations, pool generation, pilots, the rejected regression fine-tuning, and earlier model-development experiments. They are not the total project compute budget.

5.2 Selection Protocol

Following [1, Section 3], all architectural, prior, and hyperparameter decisions were made on datasets disjoint from TabArena [3]: **D1** (1,536 held-out synthetic classification tasks: 384 each from graph v3, graph v4, rule v1, and tree v4); **D2** (125 synthetic mechanism probes); **D3** (the 55 OpenML-CC18 classification datasets outside TabArena, 5-fold stratified CV at $\leq$ 1,000 rows, $\leq$ 100 features); **Dcat** (6 large OpenML categorical datasets outside TabArena and D3 with $>$ 2 category levels: *albert*, *porto-seguro*, *kick*, *okcupid-stem*, *sf-police-incidents*, *airlines*, evaluated at 10,000 and 25,000 training rows with 1 estimator and full context); **Scale** (15 datasets, 162 splits from 1k to 94,320 training rows) and **D4** (13 large datasets, 46 splits at 10k–100k rows); and for regression, **D1-reg** (36 synthetic tasks), **D3-reg** (23 OpenML-CTR23 regression datasets outside TabArena, 115 five-fold splits at $\leq$ 1,000 rows), and **D3-reg-L** (the 15 D3-reg datasets with $\geq$ 4,000 finite rows, 30 splits up to 40,000 training rows). TabArena was reserved for final evaluation, with one exception: during the wide-table fine-tuning (Section 6.2) the first TabArena split, scored from saved predictions, entered the pre-declared early-stop and extension rules (not the gate), so it is not an independent test of that stage.

6 Results: Classification

6.1 Milestone Progression and External Benchmarks (D1, D2, D3, Dcat, Scale, D4)

Table 1 tracks the EMA checkpoints of LightPFN 1.5 across `cls_main` (30k, 60k, 90k, 120k steps) and `cls_long` (+5k, +10k, +15k steps, up to 135k total) alongside LightPFN 1.0 (30k and the released 120k), LightPFN 1.1.0 (`final_long`, 159.25k total steps), and default GBDT baselines ($n_{\text{estimators}} = 1$, ground-truth dataset categorical masks). Figure 3 plots the progression on D1, D3 and Dcat 10k across the schedule and the per-dataset Dcat breakdown against larger foundation models.

Three findings stand out from Table 1 and Figure 3; they refer to the `cls_long` 15k checkpoint, and Section 6.2 compares the final, wide-table fine-tuned classifier with it:

- **D1, D2, and D3 ($n_{\text{estimators}} = 1$).** At the final 135k checkpoint (`cls_long` 15k), LightPFN 1.5 matches LightPFN 1.1.0 on D1 (0.8777 vs. 0.8783; paired difference -0.05 points $[-0.19, +0.08]$, with $+0.03$ points on graph v4 and -0.21 points on graph v3) and improves D2 by $+1.40$ points $[+0.18, +2.89]$ (0.9531 vs.

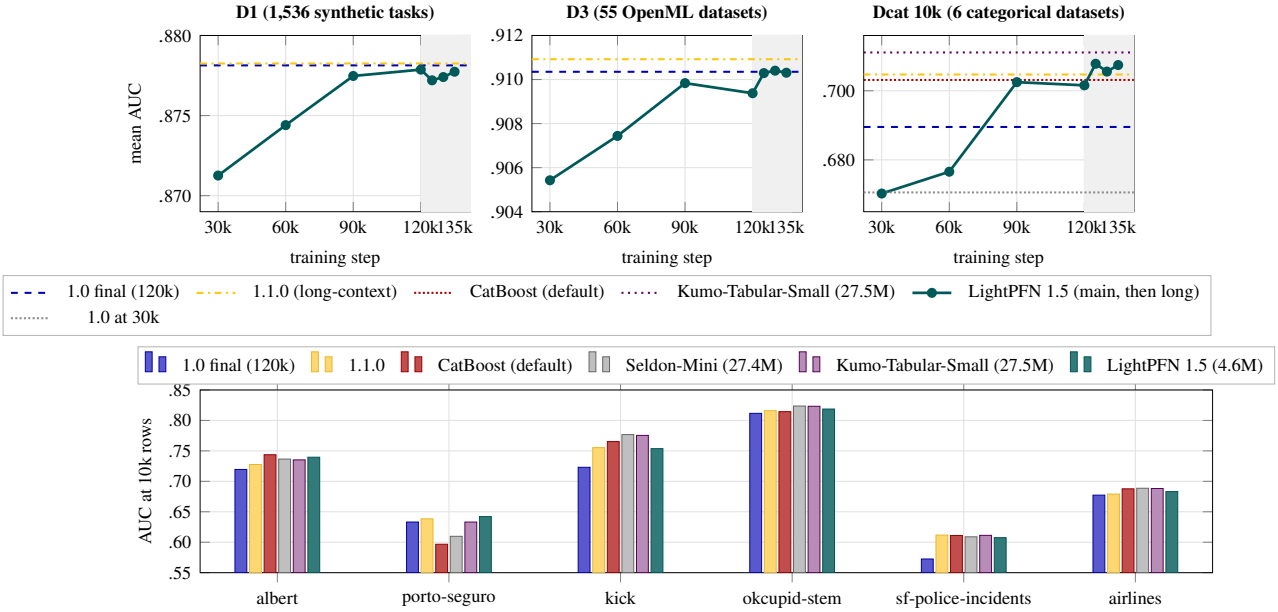


Figure 3. Classification during the 1.5 schedule (single estimator, full context). **Top:** D1, D3 and Dcat 10k mean AUC of the EMA checkpoints of `cls_main` (30k to 120k steps on at most 2,048 rows) and `cls_long` (shaded, 120k to 135k steps on up to 60,000 rows), with the 1.0 `final` (120k) and 1.1.0 `final_long` checkpoints, default CatBoost and 27.5M-parameter Kumo-Tabular-Small as references. **Bottom:** per-dataset Dcat AUC at 10,000 training rows; LightPFN 1.5 is the `cls_long` 15k checkpoint, Seldon-Mini and Kumo-Tabular-Small are 27M-parameter models with one estimator.

0.9391). On the 55 external real datasets of D3, LightPFN 1.5 reaches 0.9103 mean AUC (vs. 0.9109 for 1.1.0, paired difference -0.06 points $[-0.33, +0.15]$, 27/55 wins). Against default GBDTs on D3, LightPFN 1.5 leads default CatBoost by +0.80 points $[+0.45, +1.19]$ (46/55 wins), default LightGBM by +1.96 points $[+1.10, +3.11]$ (47/55 wins), and default XGBoost by +1.99 points $[+1.29, +2.85]$ (49/54 wins; its failed task is excluded from both sides).

- **High-cardinality categoricals (Dcat 10k and 25k).** At 30,000 steps, `cls_main` matches v1 30k on Dcat 10k (0.6703 vs. 0.6706), but between 60,000 and 90,000 steps the combined training recipe shows a jump of +2.59 AUC points to 0.7025, finishing `cls_main` (120,000 steps, before any training on $> 2{,}048$ rows) at **0.7016**, which is already +1.21 points above the 120,000-step v1 checkpoint (0.6895, 6/6 datasets) and +3.10 points above v1 30k (0.6706). With 15,000 steps of long-context training (`cls_long` 15k, vs. 39,250 long-context steps in 1.1.0), Dcat 10k reaches **0.7075** and Dcat 25k reaches **0.7134**. Across the 12 cells of Dcat (10k and 25k rows), LightPFN 1.5 beats 1.1.0 by +0.32 points $[+0.03, +0.61]$ (9/12 cells: +0.27 points at 10k, +0.36 points at 25k), beats v1 120k by +1.89 points $[+1.33, +2.47]$ (12/12 cells), and exceeds default CatBoost by +0.43 points at 10k (0.7075 vs. 0.7031, with +4.54 points on *porto-seguro* and +0.43 on *okcupid-stem*) and +0.16 points at 25k (0.7134 vs. 0.7118). At 10,000 rows with a single estimator, LightPFN 1.5 (4.61M parameters, 0.7075) has a similar mean AUC to Seldon-Mini [11] (27.4M parameters, 0.7074) and comes within 0.36 points of Kumo-Tabular-Small [7] (27.5M parameters, 0.7111).
- **Scale suite and D4 (up to 100,000 rows).** On the 162 splits of the Scale suite, LightPFN 1.5 reaches 0.9102 mean AUC (vs. 0.9097 for 1.1.0 and 0.9071 for default CatBoost). On the 46 cells of D4 (10k, 25k, 50k, and 100k training rows), LightPFN 1.5 averages **0.8200** AUC, numerically exceeding LightPFN 1.1.0 (0.8188, +0.11 points $[-0.09, +0.32]$, 29/46 wins) and default CatBoost (0.8189, +0.11 points $[-0.32, +0.57]$), while leading default LightGBM (0.7848) by +3.52 points and default XGBoost (0.7903) by +2.97 points. By training size on D4, LightPFN 1.5 leads default CatBoost at 10,000 rows (0.8172 vs. 0.8141, 13 tasks) and 25,000 rows (0.8246 vs. 0.8225, 13 tasks), and virtually ties at 50,000 rows (0.8295 vs. 0.8311, 12 tasks) and 100,000 rows (0.8025 vs. 0.8027, 8 tasks).

Benchmark (4 estimators)	Tasks	Final − `cls_long` 15k	95% CI
D3-wide AUC (points)	9	+1.93	[−0.06, +5.82]
D3-wide log loss	9	−0.048	[−0.108, −0.004]
D3 AUC (points)	55	+0.01	[−0.06, +0.09]
D1 AUC (points, 1 estimator)	1,536	+0.05	[−0.03, +0.13]
Dcat 10k AUC (points)	6	+0.07	[+0.00, +0.16]
D4 AUC (points)	13	+0.12	[+0.05, +0.21]
TabArena fold 0, relative error	38	−0.85%	[−1.83%, −0.04%]
tasks with > 60 features	9	−3.26%	[−6.40%, −0.97%]

Table 2. Wide-table fine-tuning: paired differences between the final classifier (`cls_wide` 8,500 steps) and the `cls_long` 15k checkpoint, task-level bootstrap. Negative log loss and error are better. TabArena relative error uses 1−AUC for binary and log loss for multiclass tasks, as the leaderboard.

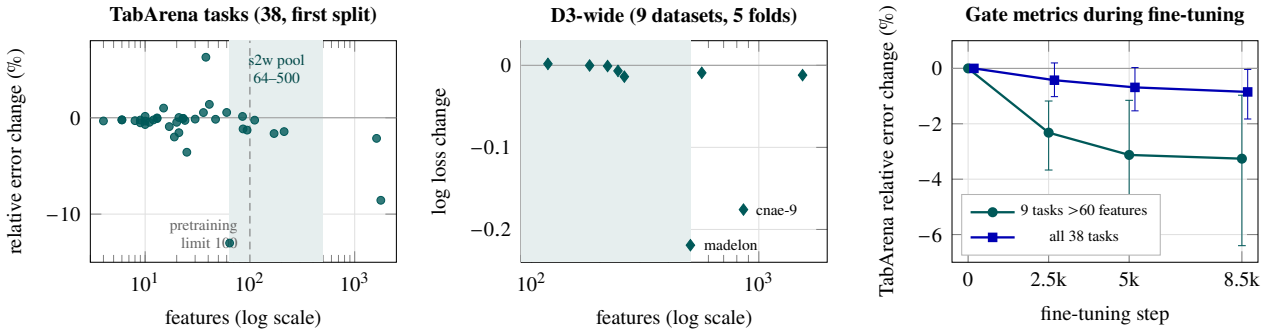


Figure 4. Wide-table fine-tuning, final classifier minus the `cls_long` 15k checkpoint (four estimators, same RTX 5090 node). **Left:** relative error change on the first split of the 38 TabArena classification tasks against their number of features; the shaded band is the 64–500 feature range of the new `s2w` pool, the dashed line the 100-feature limit of pretraining. **Middle:** log loss change on the nine D3 datasets with more than 100 features (five folds each). **Right:** TabArena relative error change, with 95% task-bootstrap intervals, at the 2,500-, 5,000- and 8,500-step milestones; these milestones were monitored during the run (Section 6.2).

6.2 Wide-Table Fine-Tuning

Table 2 compares the final classifier with the `cls_long` 15k checkpoint discussed above, with four estimators (the package default), both evaluated on the same RTX 5090 node. The gain is concentrated on wide tables: on D3-wide the log loss improves with an interval below zero, and the AUC gain (+1.93 points) has a wide interval driven by a few datasets. D3, D1, Dcat and D4 do not worsen; D4 and Dcat improve slightly. On the first split of the 38 TabArena classification tasks the relative error falls by 0.85%, and by 3.3% on the nine tasks with more than 60 features. That split was monitored at the 2,500-, 5,000- and 8,500-step milestones (it entered the pre-declared early-stop and extension rules, not the gate), so it is not an independent test. Between 5,000 and 8,500 steps every metric changed by less than its interval, so the run was not extended. With one estimator the final classifier has D3 AUC 0.9102 and Dcat 10k 0.7082, against 0.9103 and 0.7077 for the 15k checkpoint on the same node.

Figure 4 shows where the gain comes from. On the TabArena split, eight of the nine tasks with more than 60 features improve, the largest by 13.0% (64 features) and 8.6% (1,776 features); tasks with fewer than 60 features change by at most 2% except two (a 25-feature task, −3.6%, and a 38-feature five-class task, +6.3%). On D3-wide two datasets carry the log-loss gain: *madelon* (500 features, −0.219, AUC +17.4 points) and *cnae-9* (856 features, −0.176); the other seven change by less than 0.015. Most of the gain is already present at 2,500 steps.

6.3 Local TabArena-Lite Classification Evaluation

To verify the TabArena integration end to end, we ran TabArena-Lite [3] with the official pipeline and the updated LightPFN model class (registered under a temporary second name, so that the hosted 1.1 results stay in the same comparison) (default configuration, 8-fold bagging with refit, official validation protocol) on the first

Subset	Method	Pos.	Elo	95% CI	Rank	Train (s/1k)	Predict (s/1k)
Classification (38 tasks, fold 0, RTX 5090)	Kumo-Tabular (default)	1 / 115	2026	+156/−113	6.82	2.79	0.271
	LimiX-2 (default)	5 / 115	1883	+167/−113	12.13	36.88	10.120
	Seldon-Mini (default)	10 / 115	1786	+104/−83	17.25	0.58	0.040
	Kumo-Tabular-Small (default)	13 / 115	1771	+83/−58	18.13	0.82	0.092
	TabICLv2 (default)	25 / 115	1565	+78/−69	33.82	0.93	0.141
	LightPFN 1.5 final (author run)	34 / 115	1436	+74/−65	46.84	0.29	0.023
	LightPFN 1.5 before wide fine-tuning*	35 / 115	1416	+72/−66	49.05	0.29	0.024
	LightPFN 1.1 (default)	35 / 115	1423	+69/−70	48.26	0.26	0.029
	CatBoost (tuned)	40 / 115	1387	+58/−57	52.34	1087.73	0.034
	CatBoost (default)	53 / 115	1341	+50/−57	57.61	4.89	0.025
	XGBoost (default)	85 / 115	1192	+61/−73	74.97	1.88	0.121
	LightGBM (default)	93 / 115	1140	+62/−66	80.57	2.25	0.159
Regression (13 tasks, fold 0, RX 7900 XT)	Kumo-Tabular (default)	1 / 111	2668	+355/−127	3.46	2.01	0.246
	Seldon-Mini (default)	7 / 111	2337	+208/−80	10.00	0.38	0.034
	Kumo-Tabular-Small (default)	11 / 111	2247	+210/−98	12.62	0.71	0.095
	TabICLv2 (default)	30 / 111	1737	+241/−155	35.69	0.67	0.118
	CatBoost (tuned)	46 / 111	1526	+75/−88	51.00	2267.19	0.046
	LightPFN 1.5 (author run)	54 / 111	1450	+107/−103	57.31	0.21	0.033
	CatBoost (default)	59 / 111	1416	+84/−125	60.15	9.02	0.018
	LightGBM (default)	82 / 111	1217	+60/−106	77.08	2.03	0.296
	XGBoost (default)	87 / 111	1183	+100/−155	79.77	2.15	0.180

Table 3. Author-run TabArena-Lite evaluation of LightPFN 1.5 with the official pipeline and the updated LightPFN model class (first split of each dataset; AMD RX 7900 XT except the final classifier, run on an RTX 5090; compared with the hosted results of the other entries in `TabArenaContext`; **not** the official maintainers' leaderboard). Top: 38 classification datasets (115 entries). Bottom: 13 regression datasets (111 entries). Positions count every entry in the local comparison. *The `cls_long` 15k checkpoint, run with the same class on the same RTX 5090 in a separate comparison.

split of all 38 classification and 13 regression datasets: on the author's workstation (AMD RX 7900 XT GPU through ROCm, Intel i7-13700KF CPU, 8 threads), and for the final classifier and its `cls_long` reference on one rented NVIDIA RTX 5090 (8 threads). The package code was the released inference code. We emphasize that these numbers are **local, unverified TabArena-Lite estimates** compared against the hosted baseline snapshot in the TabArena package, *not* the official maintainers' leaderboard across all repeats and folds:

- On the 38 classification tasks (Table 3, top block), the final classifier completes all tasks with no failure or imputation and reaches a local Lite Elo of **1436** (+74/−65, 34th of 115 entries, mean rank 46.84) on an RTX 5090. The `cls_long` 15k checkpoint, run with the same class on the same GPU, scored 1416 (+72/−66, 35th; 1415 on the RX 7900 XT); the final classifier has lower error on 24 tasks and higher on 14. Both are level with LightPFN 1.1 on the same split (1423, +69/−70, in the comparison of the final classifier; the intervals overlap) and above every tree-based entry, including tuned CatBoost (1387) and default CatBoost (1341). This matches the development-set picture: on ordinary tables 1.5 equals 1.1, and its gains are on high-cardinality categorical columns, which are rare in TabArena.
- Median times were 0.29/0.023 s per 1k rows (training/prediction) for the final classifier on the RTX 5090 (0.29/0.024 for the `cls_long` checkpoint on the same GPU, 0.31/0.041 on the RX 7900 XT), with other CPU jobs running on the machine; they are not comparable with the official timings of 1.1, which come from the maintainers' hardware (Section 9 compares both versions on the same GPU).
- Figure 5 places these results against training time. In both local comparisons LightPFN 1.5 lies on the Pareto front of Elo against median training time per 1k rows, and in regression its training time is the lowest of all 111 entries. Its timings come from the author's GPU and the others from the hosted runs on other hardware, so this is an indication, not a verified position.
- The official evaluation by the TabArena maintainers on all splits of the 51 datasets is reported in Section 8; the local numbers above are first-split estimates on other hardware.

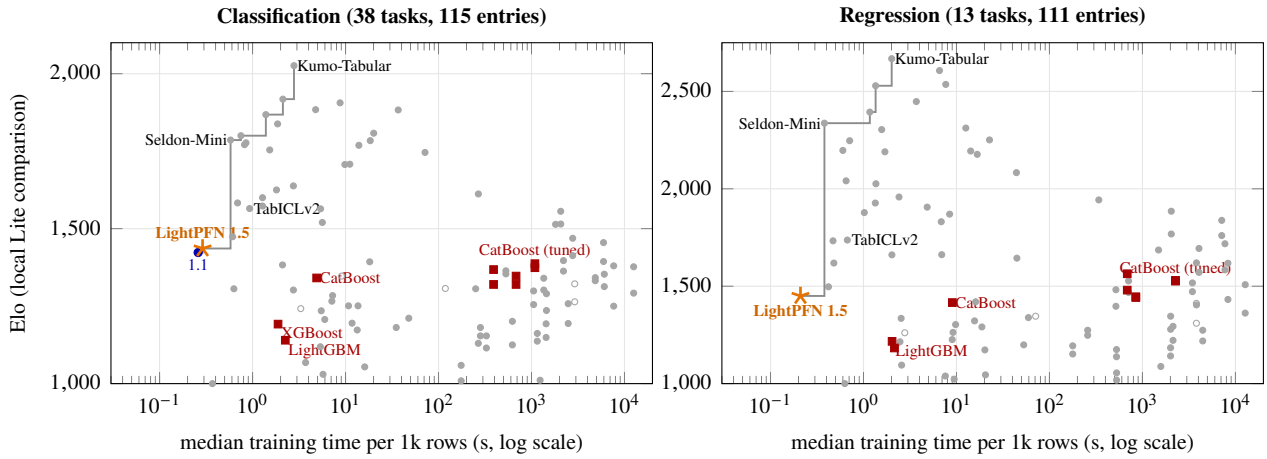


Figure 5. Local TabArena-Lite comparisons (first split; classification with the final classifier on an RTX 5090, regression on an RX 7900 XT): Elo against median training time per 1,000 rows, log scale. Grey: other entries (hollow: entries with imputed tasks); red squares: default and tuned GBDTs; blue: LightPFN 1.1; star: LightPFN 1.5. The staircase is the Pareto front over the non-imputed entries. The hosted entries were timed on other hardware.

7 Results: Regression

7.1 Progression on D1-reg, D3-reg, and D3-reg-L

Across the 39,063 steps of `reg_main` and 5,000 steps of `reg_long`, single-member evaluation at 20,000 max context shows steady improvement on the 23 real OpenML tasks of D3-reg (115 five-fold splits): mean normalized CRPS (CRPS/σ_y) drops from 0.1826 at 10k steps to 0.1811 at 20k, 0.1782 at 30k, 0.1771 at 39,063 steps, and **0.1765** at the end of `reg_long` (+5k steps), while normalized RMSE (RMSE/σ_y) drops from 0.4213 to 0.4111 at 39,063 steps and **0.4080** at +5k steps (mean R^2 rising from 0.7510 to 0.7623).

On the 36 synthetic D1-reg tasks, normalized CRPS is 0.4389 at 10k, 0.4357 at the end of main fine-tuning, and 0.4372 after the long stage; thus the short synthetic benchmark does not improve monotonically with long-context training.

Two-member inference and long-context evaluation. The initial fixed-weight study used the final EMA checkpoint with BF16 inference. D3-reg has 23 tasks with at most 800 context rows. D3-reg-L uses the 15 D3-reg sources with at least 4,000 finite rows, two random splits each, test size 25% capped at 5,000, and context capped at 40,000 rows; it is disjoint from TabArena. The wrapper's 50,000-row limit therefore retains all available context.

1. In that study, increasing the one-member context limit from 20,000 to full context leaves D3-reg unchanged and lowers mean D3-reg-L normalized RMSE from 0.4213 to 0.4183. The preceding two-member package rule, which used $\log y$ for every strictly positive target, improved relative RMSE by 1.03% [0.40, 1.73] on D3-reg and 6.72% [1.31, 14.12] on D3-reg-L versus one member at 20k. These are historical BF16 measurements, not the final package rule.
2. The final skewness-gated two-member rule, evaluated on CPU in FP32 with full available context, reaches **0.1748** normalized CRPS and **0.4042** normalized RMSE on D3-reg, and **0.1546** normalized CRPS and **0.3691** normalized RMSE on D3-reg-L. Relative to the old rule rerun on the same CPU, per-task mean RMSE changes by −0.70% [−1.52, −0.10] and −1.78% [−4.05, −0.15], respectively. The threshold was selected on these development sets, so these estimates are optimistic.
3. On D3-reg, the final rule beats default CatBoost on **22/23 tasks** (geometric mean RMSE ratio 0.791); CatBoost reaches CRPS 0.2370 and RMSE 0.4794, LightGBM 0.2546 and 0.5155, and XGBoost 0.2693 and 0.5450. Their predictive quantiles use held-out residual calibration on 20% of the context, as in the external-regression harness; these scores do not use TabArena's bagging protocol. On D3-reg-L, the final rule wins **11/15 tasks** against point-prediction CatBoost with a geometric mean RMSE ratio of **0.921**; arithmetic mean normalized RMSE is 0.3691 vs. 0.3689. Thus the mean-score comparison depends strongly on aggregation

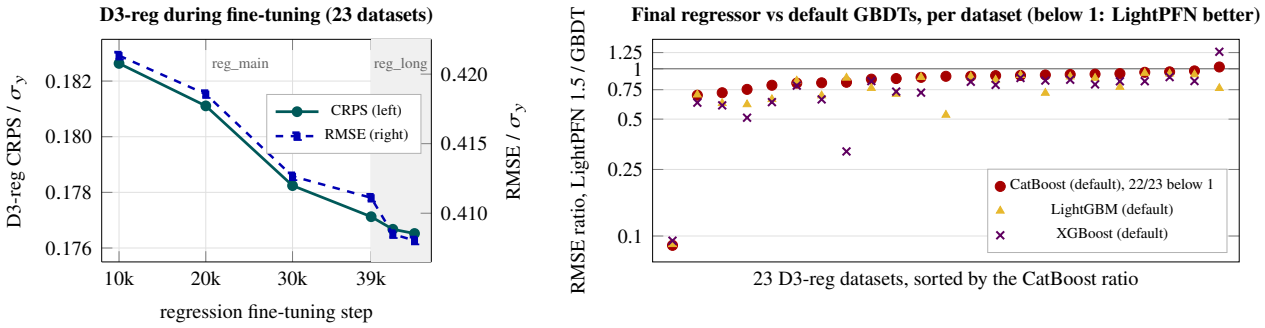


Figure 6. Regression. **Left:** D3-reg normalized CRPS (left axis) and RMSE (right axis) of the EMA checkpoints during `reg_main` and `reg_long` (shaded), one member, 20,000 context rows. **Right:** per-dataset RMSE of the released regressor (two members, final rule, CPU FP32) divided by that of default CatBoost (circles), LightGBM and XGBoost, sorted by the CatBoost ratio; log scale, below 1 favours LightPFN 1.5. Both use the 115 saved D3-reg splits, five per dataset.

and a few difficult tasks.

Figure 6 shows the progression and the per-dataset comparison with default GBDTs: the final regressor is below default CatBoost on 22 of the 23 D3-reg datasets, with ratios from 0.09 to 1.03.

7.2 A Wide-Table Regression Fine-Tuning That Failed Its Gate

After the classification result we tried the same recipe on the regressor: 5,000 steps from `reg_long` (same optimizer settings) on fresh `r2` pools with wider tables, judged on a new development set of 12 OpenML regression datasets with up to 2,000 features (D3-reg-wide), with the gate fixed before the run. The fine-tuned regressor was worse where it was meant to help: relative RMSE +1.85% [+0.59, +3.47] and CRPS +2.07% on D3-reg-wide, and it lost on D3-reg (+0.76% RMSE) and D3-reg-L (+1.66%) as well. The released regressor is therefore unchanged. Wider synthetic tables alone, at this length and learning rate, do not transfer to real wide regression tables; the causes listed in Section 7.3 (out-of-range column positions, categorical cardinality of the regression prior) remain open.

7.3 Local TabArena-Lite Regression and Analysis of the Gap to Larger TFMs

On the 13 regression tasks of TabArena-Lite (fold 0, Table 3, bottom block), the released regressor (two members with the skewness-gated second member, FP16 with FP32 decoder head, 50,000 context rows) reaches a local Lite Elo of **1450** (+107/−103, 54th of 111 entries, mean rank 57.31), above default CatBoost (1416) and far above default LightGBM (1217) and XGBoost (1183), at 0.21 and 0.033 s per 1k rows for training and prediction. Earlier development runs of the same checkpoint with older wrappers scored 1315 (one member, 20,000 context rows, BF16) and 1351 (two members with the previous rule) against a smaller set of entries, so the steps are indicative only. Tuned CatBoost (1526) and the larger foundation models remain clearly ahead. In this same final local comparison, the point-estimate gaps are +233 Elo versus default LightGBM, +267 versus XGBoost, and +34 versus CatBoost. The CatBoost intervals overlap widely; the point estimate is not a conclusive superiority claim.

Honest comparison with larger foundation models and CatBoost. While LightPFN 1.5 is competitive with default GBDTs in regression, it trails larger tabular foundation models by a wide margin: on TabArena-Lite regression, 27.5M-parameter Kumo-Tabular-Small [7] reaches Elo 2247 (mean rank 12.62) and TabICLv2 [10] reaches Elo 1737 (mean rank 35.69), and on D3-reg Kumo-Tabular-Small ($n = 1$) achieves 0.160 normalized CRPS and 0.3775 normalized RMSE (beating LightPFN 1.5 on 21 of 23 tasks). To understand where our 4.77M-parameter regressor loses, we conducted a task-by-task error analysis across the 13 TabArena-Lite regression tasks, D3-reg, and D3-reg-L:

1. **Concentration of losses against default CatBoost.** Across the final 13-task Lite run, LightPFN 1.5 beats default CatBoost in geometric mean RMSE ratio (0.9723, 8/13 tasks), with strong wins on *airfoil_self_noise* (0.775) and *physiochemical_protein* (0.895). Its largest relative losses are *QSAR-TID-11*

(1.067), *Food_Delivery_Time* (1.062), and *superconductivity* (1.019). These are descriptive comparisons; no inference variant was selected on these tasks.

2. **Wide tables beyond the pretraining feature cap ($m = 1024$ on *QSAR-TID-11*).** Pretraining capped the number of features at $m \leq 100$. In `RowStage` and `RowRefinement`, 1D rotary position embeddings (RoPE) [12] are applied along the column index $0, 1, \ldots, m-1$, and cross-attention over columns uses unscaled softmax (`scaling=False`, unlike the $\log n$ scaling in `ICLStage`). On *QSAR-TID-11* ($n_{\text{train}} = 3{,}828$, $m = 1{,}024$ fingerprint features), column indices $100 \ldots 1023$ receive out-of-distribution RoPE angles never seen during training and unscaled cross-attention entropy over 10× more columns.
3. **Long-and-wide cell cap starvation ($n_{\text{train}} = 14{,}175$, $m = 81$ on *superconductivity*).** In the long-context stage (`s4cap`), pool generation capped `--max-cells` at 800,000 ($m \leq 800{,}000/n$, so generated tasks with $n > 10{,}000$ rows always had $m < 80$ features), and training micro-batching (`--max-cells 1380000`, `--row-cost 10`, `--fit-rows`) subsampled rows whenever $n(m+10) > 1.38\times10^6$ ($n \leq 12{,}545$ at $m = 100$). Thus the joint regime of $n > 14{,}000$ rows and $m > 80$ features was never seen at full size during `reg_long`.
4. **Prior categorical cardinality cap in `graph v3` and noisy high-cardinality IDs.** While classification pretraining used `graph v4` (Zipf-distributed levels up to 256) and `--cat-mask-drop 0.25`, regression fine-tuned the trunk and regression head for 44,063 steps on `graph v3` (`modern=False`, `--cat-mask-drop 0`). An audit of the upstream `tabicl` graph v3 categorical size sampler revealed that because the flag `use_corrected_cat_sizes` defaults to `False`, `graph v3` ignores `max_cat_size=200` and caps categorical cardinality at $\min(10, d_{\max} + 1) = 10$ ($2 \ldots 9$ levels). Consequently, during regression fine-tuning the model never saw a categorical column with more than 9 levels. On *Food_Delivery_Time* ($n_{\text{train}} = 30{,}300$), column `Delivery_person_ID` has 1,320 levels ($\sim$ 23 rows per level) and negative out-of-fold target-encoding R^2 (-0.039 at smoothing $\alpha = 1.0$; -0.001 even at $\alpha = 100$): the diagnostic encoding did not show useful predictive signal in that split. A noisy high-cardinality ID is therefore a plausible source of overfitting, rather than necessarily a missed signal. In the diagnostic split, dropping it reduces HistGradientBoosting RMSE to 7.480, close to CatBoost 7.419 (the older LightPFN run scored 7.891). This is an HGB intervention, not evidence that dropping the ID fixes LightPFN.
5. **Target transformation gating and extreme context target outliers (D3-reg-L).** In the earlier development regressor, the second member used $\log(y)$ whenever $\min(y) > 0$. Stratifying positive-target tasks by context target skewness suggests using $\log(y)$ when target skewness is high (skew ≥ 2.0, such as *brazilian_houses* and *diamonds*): on the 10 positive D3-reg tasks with skew < 2.0, using $-y$ (`flip2`) instead of $\log(y)$ lowers RMSE by -1.55% (8/10 wins), and on the 5 positive D3-reg-L tasks with skew < 2.0 (*naval_propulsion*, *fps_benchmark*, *kin8nm*, *wave_energy*, and *abalone*), `flip2` lowers RMSE by -4.88% (4/5 wins, including -13.3% on *naval_propulsion* and -7.8% on *fps_benchmark*). When $\sigma_y/\mu_y \ll 1$, $\log(y)$ is nearly linear after standardization ($z_{\log y} \approx z_y$), losing the sign-symmetry diversity from $z_{-y} = -z_y$, while when $y_{\min} \to 0^+$ with low skewness, $\log(y)$ stretches the lower tail into artificial negative skewness. Conversely, on *brazilian_houses* in D3-reg-L, two duplicate training rows with $y = 1{,}120{,}000$ (300× the median, context skewness 68.98) pass unclipped through $z_y = (y - \mu_y)/\sigma_y$ ($z_y = 81.6$, $z_y^2 = 6{,}652$) into `RetrievalDecoder.moments`, while input values are soft-clipped. This is a plausible failure mechanism, not an isolated causal explanation of the D3-reg-L loss. Following this analysis, the released regressor uses $\log(y)$ only when $\min(y) > 0$ and the adjusted sample skewness of the context targets is at least 2, and $-y$ otherwise. Re-running the full development sets on CPU in FP32 with fixed weights, this rule lowers normalized RMSE by 0.70% [0.10, 1.52] on D3-reg and 1.78% [0.15, 4.05] on D3-reg-L relative to the previous rule, and CRPS by 0.72% and 1.55%. The threshold was chosen on these same development tasks, so the gains are optimistic; the TabArena-Lite run in Table 3 uses the new rule.

8 Official TabArena Leaderboard

The TabArena maintainers ran `lightpfn` 1.5.1 (the released weights with the inference fixes and speedups of Section 9) on all 816 splits of TabArena-v0.1 [3]: 38 classification and 13 regression datasets, official 8-fold bagging with refit, default configuration (four classification estimators, two regression members, no search space),

View (all splits)	Datasets	Position	Elo (95% CI)	Train / predict (s per 1k)	Default CatBoost
All tasks	51	**31 / 99**	**1447** (+53/−61)	0.23 / 0.019	43rd, 1383
All tasks, Lite (first split)	51	30 / 99	1433 (+54/−58)	0.24 / 0.019	45th, 1358
Classification	38	**28 / 103**	**1468** (+63/−62)	0.24 / 0.019	41st, 1388
Binary	30	29 / 103	1451 (+67/−62)	0.19 / 0.014	36th, 1408
Multiclass	8	25 / 103	1559 (+146/−98)	0.37 / 0.024	59th, 1326
Regression	13	**47 / 101**	**1431** (+129/−99)	0.16 / 0.024	51st, 1410
All tasks, small datasets		27 / 99	1462 (+73/−60)	0.29 / 0.024	49th, 1323
All tasks, medium datasets		49 / 99	1446 (+116/−116)	0.08 / 0.015	32nd, 1584

Table 4. Official TabArena leaderboard of 11 November 2026, LightPFN 1.5.1, imputed entries excluded. Elo is computed among the entries of each view and is not comparable across rows. Times are the leaderboard medians per 1,000 rows. With imputed entries included (the default view of the page), LightPFN is 31st of 110 with Elo 1426 (+49/−55).

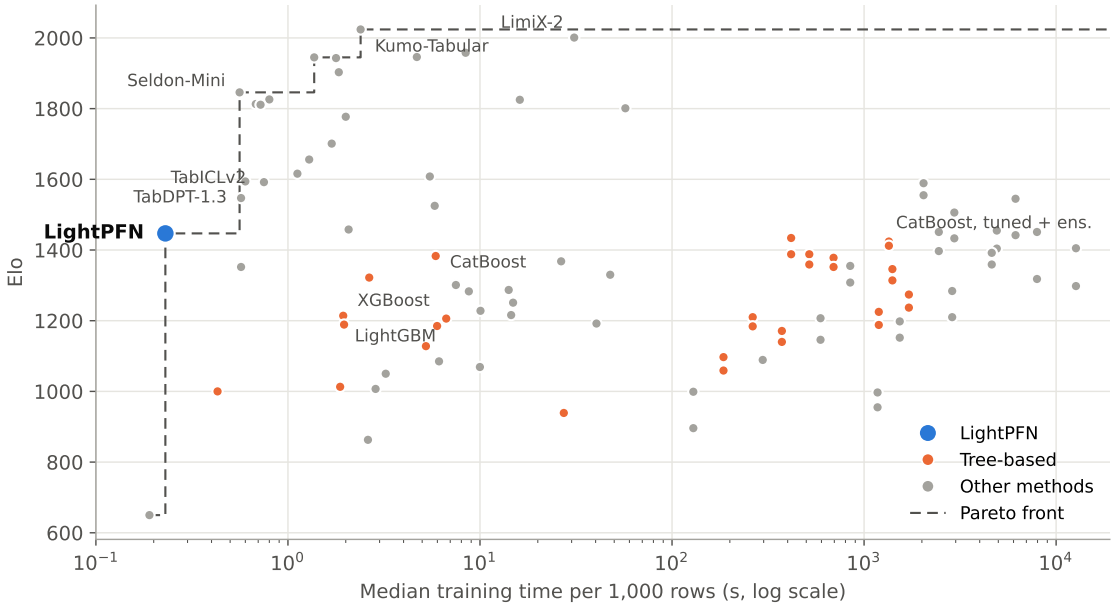


Figure 7. Official TabArena leaderboard of 11 November 2026, all 51 datasets, all splits, imputed entries excluded: Elo against median training time per 1,000 rows (log scale, tuned entries include their search time). Blue: LightPFN 1.5.1; orange: tree-based entries; grey: other entries; dashed: Pareto front. GPU methods were timed on a GPU and tree-based methods on a CPU.

one RTX PRO 6000 GPU. The result is verified, nothing is imputed, and it replaced the LightPFN 1.1 entry on 11 November 2026 (Space data commit `e0723ee`). Table 4 lists the views with imputed entries excluded.

Over all 51 datasets and in classification, the Elo point estimate of LightPFN 1.5.1 is above every tree-based entry, including tuned and ensembled LightGBM (1434 over all datasets) and CatBoost (1424); the intervals overlap those of the tuned gradient-boosting entries and of default CatBoost, so against them this is not a clear win, while default XGBoost (1214) and LightGBM (1189) are more than 200 points lower with separate intervals. In regression, which LightPFN did not support before 1.5, it is above default CatBoost (1410, overlapping intervals) and below the tuned gradient-boosting entries (1442 to 1522); the larger foundation models are far ahead (Kumo-Tabular-Small 2130, Seldon-Mini 2150, TabICLv2 1712), in line with Section 7.3. As with 1.1, the weak spot is the medium datasets (49th of 99, default CatBoost 32nd). LightPFN is on the Pareto front of Elo against median training time in every view of Table 4 and against median prediction time in every view except the medium datasets (Figure 7): over all tasks only KNN trains faster and only EBM predicts faster, both with much lower Elo, and in regression its training time is the lowest of all 101 entries. On the same hardware the maintainers measured 0.290 and 0.019 s per 1,000 rows for 1.5.0 against 0.235 and 0.019 s for 1.5.1 over all tasks, with the same Elo.

Dataset (rows × features)	1.1.0 FP32 (train / pred)	1.5 Auto (train / pred)	Train speedup	Predict speedup
`diabetes` (768×8)	0.261 / 0.220	0.192 / 0.156	1.36×	1.41×
`credit-g` (1,000×20)	0.207 / 0.177	0.191 / 0.133	1.08×	1.33×
`MIC` (1,699×111)	0.601 / 0.479	0.386 / 0.271	1.56×	1.77×
`splice` (3,190×60)	0.318 / 0.272	0.225 / 0.151	1.42×	1.80×
`heloc` (10,459×23)	0.220 / 0.224	0.094 / 0.089	2.33×	2.52×
`coil2000_insurance_policies` (9,822×85)	0.433 / 0.408	0.253 / 0.230	1.71×	1.77×
`credit_card_clients_default` (30,000×23)	0.388 / 0.433	0.120 / 0.133	3.24×	3.25×
`Amazon_employee_access` (32,769×9)	0.372 / 0.442	0.094 / 0.128	3.93×	3.45×

Table 5. Classifier inference under the local 8-fold, 4-estimator protocol on the same RTX 5090 (CUDA 13.0, 8 CPU threads). Training includes fit and validation prediction. Times are summed over the eight folds, then divided by the outer training-split size for train or the fixed held-out test size for predict, not by the sum of the individual fold sizes. Speedups compare original 1.1.0 FP32 against the 1.5 architecture (measured with the `cls_long` 15k weights, which have the same shapes and cost as the final classifier) with the measured `auto` package path (8,192-cell FP16 threshold). There is no additional refit in this timing surrogate.

9 Inference Speed

9.1 FP16 Autocast, Shared Weight Views, and Compiler-Free Fusions

In TabArena's evaluation protocol (`8x1` with $n_{\text{estimators}} = 4$, denoted `8x4`), the official integration uses fold models and a refit model; the timing surrogate below measures the eight fold models only, each running four estimators. Profiling LightPFN 1.1.0 on GPU showed that three factors dominated runtime: (1) all matrix multiplications and attention kernels ran in FP32; (2) constructing a fresh FP16 view for each fold initially added overhead despite sharing the folded FP32 network; and (3) elementwise sequences (parameterless RMSNorm, RoPE rotations, and `SoftmaxScaling` modulation) launched dozens of small memory-bound kernels per forward pass.

In LightPFN 1.5 (`lightpfn/sklearn.py`, `lightpfn/precision.py`, `lightpfn/model/layers.py`), we optimize inference without `torch.compile` or custom C++/Triton extensions:

1. **Default FP16 on CUDA/ROCm with silent per-block FP32 fallback (`precision="auto"`).** Both `LightPFNClassifier` and `V15Regressor` accept `precision` in {`"auto"`, `"fp32"`, `"fp16"`} (default `"auto"`). On CPU and Vulkan, `"auto"` selects `"fp32"`, executing the exact same operations as before. On CUDA and ROCm devices with fast half-precision tensor cores, `"auto"` selects `"fp16"` whenever the context contains at least `FP16_MIN_CELLS` = 2,048 cells ($n_{\text{ctx}} \times m \geq 2{,}048$), keeping tiny tables in FP32 where kernel launch latency dominates. Within the FP16 path, column statistics, rank normalization, category target statistics, retrieval moments, and final logit/probability accumulation remain in FP32/FP64. If an encoded context group or query chunk produces a non-finite value in FP16, the wrapper increments `n_precision_fallbacks_`, logs a debug message, and silently recomputes that group or chunk in FP32 using the original FP32 network.
2. **Shared half-precision weight view (`shared_fp16_view`).** Under CUDA FP16 autocast, PyTorch casts FP32 linear weights to FP16 on every call. Instead, the helper `shared_fp16_view(net)` in `lightpfn/precision.py` builds a shallow view of the folded network in which only the `Block` linear weights ($W_q, W_{kv}, W_o, W_1, W_2$) are pre-cast once to FP16, and caches the view in a weak-key dictionary keyed by the source network with a parameter-pointer signature. Across the 8 folds of a TabArena task, all fold models share the same cached FP16 view without redundant copies or casts.
3. **Compiler-free hand fusions.** The inference path caches the context-length-dependent `SoftmaxScaling` base (up to 32 entries per module), caches contiguous RoPE tables, and uses complex multiplication for interleaved FP32/FP64 RoPE. Reduced-precision RoPE keeps the sine/cosine path. Native parameterless RMSNorm is enabled only for eligible CUDA/ROCm autocast tensors below backend-specific size limits; explicit FP32 and CPU keep the original normalization arithmetic. These are inference-only changes; there is no `addmm`/`addcmul` rewrite of the scaling MLP.

GPU and CPU speed results. Table 5 reports eight real tasks from 768 to 32,769 rows. The geometric-mean speedups relative to original 1.1.0 on the same RTX 5090 are 1.89× **for training** (fit plus validation prediction; range 1.08–3.93×) and 2.04× **for prediction** (range 1.33–3.45×). The measured threshold was 8,192 cells; the package now uses 2,048 after a separate timing grid. These local ratios are not official leaderboard timings.

- **RX 7900 XT (ROCm 7.2).** The earlier eight-task probe, before shared-view and final cache changes, measured median training/prediction speedups of 1.76 × /1.75× for the FP16 API path across both 1.1 and the new classifier. This is a different experiment from the final RTX 5090 package comparison.
- **Prediction differences.** The final NVIDIA check covered 12 D3 tasks, five folds each: mean AUC change rounded to 0.0000 points, maximum absolute per-split change 0.021 points, mean log loss change $+1.3 \times 10^{-5}$, and minimum argmax agreement 99.5%. The eight-task timing CSV records nonzero probability and occasional argmax changes. There is no full-D3/Dcat exact-prediction guarantee for FP16. With the released classifier on the RX 7900 XT, `precision="auto"` changed probabilities by at most 7.4×10^{-3} on the nine tables above, with AUC equal to four decimals except the ten-class table (0.9492 vs. 0.9493, argmax agreement 99.6%); the FP32 range calibration of the 21 workloads found a largest learned activation of 115.2 (a margin of 569× to the FP16 limit) and no FP16 matrix-product input within 64× of it, and the shared half-precision weight view and the FP32 fusions were bit-exact against the unfused path at the same precision.
- **CPU speed.** With the `cls_long` 15k weights (same architecture as the final classifier), explicit FP32 and 8 threads on the local workstation, the quiet before/after cache benchmark measured 81.93 → 80.39 ms at 256 × 6 and 888.57 → 878.35 ms at 1,500 × 50 (four estimators, median of 11 paired repeats, identical probabilities). These small differences are noisy and do not establish a substantial CPU acceleration. A separate earlier single-estimator benchmark of LightPFN 1.1.0, not the final 1.5 classifier, was roughly 2.7–5.6× faster than Seldon-Mini on the tested i7-13700KF workloads.
- **CPU regression.** On *diamonds* (53,940 × 9), an 8-fold surrogate using two members and 47,197 or 47,198 context rows per fold measured 1.71 s/1k fit rows and 1.84 s/1k predicted rows (1.97 s/1k fit+validation rows) on an i7-13700KF with 4 threads. Auto selects FP32 on CPU; this was one pass with the older positive-target second-member rule, without the final refit.
- **GPU regression.** With the older positive-target rule, the 8-fold surrogate on the four largest Lite regression datasets measured FP16-vs-FP32 fit speedups of 2.3–5.5× on RTX 5090 (0.0054–0.0133 s per 1k fit rows) and 2.7–7.9× on RX 7900 XT; prediction speedups were 1.6–2.1× and 2.2–3.6×, respectively. At 50k context rows and 36 features, peak allocated memory was 2.93 GiB. Across D3-reg and D3-reg-L on each GPU, task-macro relative RMSE and CRPS differed from FP32 by less than 0.02%, with no recorded fallback. This is aggregate metric agreement, not a bound on every prediction. The earlier BF16 path was about 0.5–0.6% worse on D3-reg-L.

9.2 Vulkan Backend Status

LightPFN includes a vendor-agnostic GPU backend (`lightpfn/vulkan/`) written in WGSL and compiled to SPIR-V at startup via `wgpu`, supporting NVIDIA, AMD, and Intel GPUs on Linux and Windows without CUDA or ROCm [1, Section 11.1]. LightPFN 1.5 extends the backend to the new models: categorical Fourier features, the ordered-statistics projection, regression labels, task tokens, and decoder moments run in WGSL kernels; category statistics, target normalization, and the quantile head stay on the host. On the RX 7900 XT, mixed-table classification tests up to 12,000 context rows had a maximum absolute probability difference of approximately 2.6×10^{-5} versus CPU FP32, with identical argmax. Regression probes up to 40,000 context rows compared the 0.1, 0.5, and 0.9 quantiles: maximum absolute difference divided by the maximum absolute CPU quantile was approximately 5.5×10^{-5}. This is not a pointwise relative-error bound for all 999 quantiles, and those GPU probes used the older positive-target second-member rule. Larger-context probe workloads were approximately 3–7× faster than CPU; the smallest classification case was slower (0.53 vs. 0.36 s). NVIDIA Vulkan received correctness tests and one smoke comparison, while Intel drivers were not benchmarked here. These measurements used the `cls_long` 15k classifier. The released weights were checked again through the public API on the RX 7900 XT against CPU FP32: on nine classification tables (mixed categorical columns up

to 12,000 context rows, ten classes, and 100 to 500 features) Vulkan differed by at most 2.4×10^{-5} in probability and ROCm FP32 by at most 9.2×10^{-6}, with identical argmax; on six regression tables point predictions differed by at most 5.6×10^{-6} relative (Vulkan) and 3.3×10^{-6} (ROCm FP32). The Vulkan test suite passed on the same GPU (99 tests, 2 skipped).

10 Limitations and Next Steps

We highlight four limitations of the present work and concrete next steps:

1. **Regression gap to 27M+ tabular foundation models.** While the 4.77M-parameter LightPFN 1.5 regressor beats default LightGBM, default XGBoost, and (in geometric mean RMSE ratio on D3-reg, D3-reg-L, and TabArena-Lite) default CatBoost, it trails 27.5M-parameter Kumo-Tabular-Small and TabICLv2 on every TabArena-Lite regression task (1.099 and 1.048 geometric mean RMSE ratios; Section 7.3). Model capacity ($\sim$ 4.8M vs. 27.5M parameters) and prior coverage are plausible contributors, but these comparisons do not isolate their causal effects. Three hypotheses for future controlled tests are: (a) switching the regression base graph prior from `graph v3` (which capped categoricals at $\leq$ 9 levels because `use_corrected_cat_sizes=False`) to `graph v4` (up to 256 levels) with adaptive shrinkage on high-cardinality noisy IDs; (b) raising the cell cap in `s4cap` pool generation and `reg_long` micro-batching so the model sees joint ($n > 15{,}000, m > 80$) tables, and adding column-index scaling or feature subsampling for wide tables ($m > 100$); and (c) winsorizing or robustly scaling extreme context target outliers before computing retrieval moments $[z, z^2]$ (the skewness gate for the $\log(y)$ member is already in the released regressor).
2. **Single-seed pretraining and ablations.** While key prior comparisons (the 6-arm class-share/graph-v4 test in Section 3.1, the regression head pilot, and the `r2` regression prior A/B test in Section 3.3) included two-seed comparisons (the class-share contrast itself used one seed), the 8,000-step lookup ablation, the 30,000-step B4 vs. A3 gate, and the final 135,000-step classification and 44,063-step regression runs were executed with a single training seed due to compute budget constraints.
3. **Inference choices selected on development sets.** The regression second-member rule and the FP16 cell threshold were chosen on the same development tasks used to report their effect (D3-reg, D3-reg-L, and the eight speed tasks), so those gains are optimistic estimates. D1/D2/D3/Dcat/Scale/D4 were repeatedly consulted during development; their confidence intervals condition on the trained checkpoint and do not measure training-seed variability. Intervals over the 12 Dcat cells treat two sizes from the same dataset as separate cells and can be optimistic.
4. **One official benchmark, wide intervals.** The official TabArena results (Section 8) cover all splits, but their Elo intervals overlap those of tuned gradient boosting and default CatBoost, and the 13 regression and the medium datasets give wide intervals. The numbers in Table 3 are local TabArena-Lite runs (first split of each dataset) on the author's hardware.

11 Availability, Licensing and Reproducibility

LightPFN 1.5 code and pretrained weights (classification checkpoint `cls_wide/ema_step008500.pt` and regression checkpoint `reg_long/ema_step005000.pt`) are released under the **Apache 2.0 license**: the package `lightpfn` 1.5.1 on PyPI (`pip install lightpfn`; 1.5.0 has the same weights), the code at https://github.com/GioOtto/LightPFN (tag `v1.5.1`), and the weights in the `classifier/` and `regressor/` folders of the Hugging Face repository https://huggingface.co/ueuegio/LightPFN, loaded at an immutable revision recorded in the package. The 1.1 weights remain at their original pinned revision. Both checkpoints are pretrained exclusively on synthetic tasks generated by the SCM graph and rule generators, with the `r2` observation recipe for regression. No pretrained weights, predictions, logits, or generated datasets from another tabular foundation model (such as TabPFN, TabICL, Kumo-Tabular, Seldon, LimiX, Mitra, or EXAONE) were used at any point in training or data generation; external pretrained models were used strictly as evaluation baselines. Synthetic generator code imports the TabICL [9] prior implementation, which is not included in the inference wheel; this is code reuse, not use of pretrained-model outputs. The PyPI wheel contains infer-

ence code only and has the following direct dependencies (`torch`, `numpy`, `scikit-learn`, `huggingface-hub`, `safetensors`, and optionally `wgpu` for Vulkan).

Declaration of generative AI and AI-assisted technologies

During the preparation of this work the author used generative AI tools as coding and writing assistants: to write and review parts of the code, to run and analyse experiments, and to draft and edit the text of this report. The author reviewed and edited the results and the text as needed and takes full responsibility for the content of this report.

References

[1] G. Ottoboni. *A Sling Against Giants: LightPFN, a 4.6M-parameter tabular in-context classifier designed to stay small*. LightPFN Technical Report, version 1.2, October 2026.

[2] T. Chen and C. Guestrin. XGBoost: a scalable tree boosting system. In *KDD*, 2016.

[3] N. Erickson, L. Purucker, A. Tschalzev, D. Holzmüller, P. M. Desai, D. Salinas, and F. Hutter. TabArena: a living benchmark for machine learning on tabular data. 2025.

[4] N. Hollmann, S. Müller, L. Purucker, A. Krishnakumar, M. Körfer, S. B. Hoo, R. T. Schirrmeister, and F. Hutter. Accurate predictions on small data with a tabular foundation model. *Nature*, 637:319–326, 2025.

[5] K. Jordan, Y. Jin, V. Boza, J. You, F. Cesista, L. Newhouse, and J. Bernstein. Muon: an optimizer for hidden layers in neural networks. Technical blog post, 2024.

[6] G. Ke, Q. Meng, T. Finley, T. Wang, W. Chen, W. Ma, Q. Ye, and T.-Y. Liu. LightGBM: a highly efficient gradient boosting decision tree. In *NeurIPS*, 2017.

[7] NVIDIA. Kumo Tabular, reference implementation in structured-data-models, 2026. https://github.com/NVIDIA/structured-data-models.

[8] L. Prokhorenkova, G. Gusev, A. Vorobev, A. V. Dorogush, and A. Gulin. CatBoost: unbiased boosting with categorical features. In *NeurIPS*, 2018.

[9] J. Qu, D. Holzmüller, G. Varoquaux, and M. Le Morvan. TabICL: a tabular foundation model for in-context learning on large data. In *ICML*, 2025.

[10] TabICLv2: technical report and reference implementation (tabicl 2.2.0), 2026.

[11] Seldon and Seldon-Mini, tabular foundation models: TabArena leaderboard entries and model class (TabArena pull request 701), 2026. https://github.com/autogluon/tabarena.

[12] J. Su, M. Ahmed, Y. Lu, S. Pan, W. Bo, and Y. Liu. RoFormer: enhanced transformer with rotary position embedding. *Neurocomputing*, 568:127063, 2024.

A Hyperparameters and Parameter Breakdown

Component	LightPFN 1.5 Specification
Base trunk (B4)	4,603,088 parameters: cell dim 64, 2-layer ISAB column stage (64 inducing tokens), $R = 3$ summary row refinement ($K = 4$) + second ISAB column stage, 3-layer row compression (4 readout tokens → dim 256), 7-block ICL transformer (dim 256, 8 query heads, 1 KV head, 16 thinking tokens, $\log n$ scaling), 4-head retrieval decoder (16 label slots).
Categorical adapter	+6,384 parameters (`cat_adapter=True`): `cat_freq` (48), `cat_fourier` (2,048), `CatStatsEmbed.ts` (4,096), `CatStatsEmbed.cnt` (192); ordered target statistics in context (smoothing $\alpha = 1.0$), full context lookup for queries.
Classifier total	**4,609,472 parameters** (`LightPFNClassifier`, `cls_wide/ema_step008500.pt`).
Regression head	+163,047 parameters (`head="quantiles"`): `col_label` (128), `refine_label` (128), `icl_label` (512), `task` token (256), `QuantileHead` MLP Linear$(258, 128)$ → GELU → Linear$(128, 999)$ (162,023); single-SDPA fused retrieval moments $[\hat{\mu}, \hat{\sigma}]$.
Regressor total	**4,772,519 parameters** (`V15Regressor`, `reg_long/ema_step005000.pt`).
`cls_main` stage	120,000 steps ×64 tasks/step (7.68×10^6 tasks, 4.03M unique); 90% graph v4 + 10% rule v1 at $p_{\text{bin}} = 0.55$; 256–2,048 rows, ≤ 100 features; `--cat-mask-drop 0.25`; warmup 2,000, cosine LR $10^{-3} \rightarrow 10^{-4}$; 800,000 cells/GPU; 12.51 h on 4× RTX 5090.
`cls_long` stage	15,000 steps ×57.6 avg tasks/step (8.64×10^5 tasks) from `cls_main`; 15% `s1b` (256–2,048 rows), 35% `s2cap` (1,024–10,240 rows), 50% `s4cap` (4,096–60,000 rows, ≤ 800k cells); warmup 200, cosine LR $10^{-4} \rightarrow 10^{-5}$; $n(m + 10) \leq 1.38 \times 10^6$ with `--fit-rows`; 7.08 h on 4× RTX 5090.
`cls_wide` stage	8,500 steps from `cls_long` 15k (step counter reset); 8% `s1b`, 17% `s2cap`, 25% `s4cap`, 50% `s2w` (1,024–10,240 rows, 64–500 features, ≤ 10^6 cells); same optimizer settings as `cls_long`; about 5.0 h of logged step time on 4× RTX 5090.
`reg_main` stage	39,063 steps ×64 tasks/step (2.50×10^6 tasks, 1.37M unique) warm-started from `cls_long` EMA 15k; 100% `r2` graph v3 (`filter,noise,tails,discrete`); warmup 200, cosine LR $10^{-3} \rightarrow 10^{-4}$; 400,000 cells/GPU; `--cat-mask-drop 0`; 4.61 h on 4× RTX 5090.
`reg_long` stage	5,000 steps ×57.6 avg tasks/step (2.88×10^5 tasks) from `reg_main`; 15% `r2_s1b`, 35% `r2_s2cap`, 50% `r2_s4cap`; warmup 200, cosine LR $10^{-4} \rightarrow 10^{-5}$; $n(m + 10) \leq 1.38 \times 10^6$ with `--fit-rows`; 2.46 h on 4× RTX 5090.
Inference defaults	Classifier: $n_{\text{estimators}} = 4$, max_context = 20,000, precision = "auto" (FP16 on CUDA/ROCm when $n_{\text{ctx}} m \geq 2{,}048$, shared FP16 weight view, silent per-block FP32 fallback). Regressor: $n_{\text{estimators}} = 2$ (y and $\log y$ if $\min y > 0$ and skewness ≥ 2, else $-y$; batched $B = 2$), same `precision` policy, max_context = 50,000, trapezoidal quantile mean for point predictions.

Table 6. Summary of LightPFN 1.5 architecture, parameter counts, five-stage training recipe, and inference defaults.